Reliability Modeling and Analysis of Hospital Information System Based on Microservices

Zheng Liu[1], Huiqun Yu[1,*], Guisheng Fan[1,*], Liqiong Chen[2]
1. Department of Computer Science and Engineering, East China University of Science and Technology, Shanghai, China
2. School of Computer Science and Information Engineering, Shanghai Institute of Technology, Shanghai, China
y10180300@mail.ecust.edu.cn, yhq@ecust.edu.cn, gsfan@ecust.edu.cn, lqchen@sit.edu.cn
*corresponding author

2021 IEEE International Conference on Progress in Informatics and Computing (PIC) | 978-1-6654-2655-8/21/$31.00 ©2021 IEEE | DOI: 10.1109/PIC53636.2021.9687027

***Abstract*—In recent years, modern hospital has a large scale, complex relationship, and hospital information system (HIS) due to rapid development of computer networks. But there is still a big gap for reliable use of clinical information and management system, especially in terms of fault prevention. The microservice architecture has great advantages for development and delivery of complex system. This paper proposes a novel microservice reliability model (MSRM) for HIS based on the formalism of Predicate Petri net (PrT net). First, microservice reliability requirement design is given and PrT net is used to model the reliability of microservice, and the corresponding syntax and semantics are also presented. Then the redundancy and circuit breaker is designed by using PrT net, a composition strategy is proposed and the reliability of microservices is analyzed qualitatively and quantitatively. Based on the constructed MSRM, the correctness of PrT net modeling and effectiveness of the strategies have been proven theoretically. Finally, a public healthcare case is used to explain modeling process, and verify the effectiveness of proposed method. Experimental results show that the strategy for HIS microservice reliability is effective.**

Keywords—Hospital Information System, Microservice Architecture, Software Reliability, Petri Net

I. INTRODUCTION

Hospital information system (HIS) is an information system that integrates medical science, information, management, computer and other disciplines. It is a platform that can collect, store, process, retrieve and communicate the patient's medical care information, and meet the requirements of patient, resulting in good social and economic benefits [1]. In order to meet the needs of advanced public health care development, the medical system as technical support and infrastructure for modern hospital operation has been developed [2]. The purpose is to strengthen hospital management, improve hospital work efficiency and medical quality by scientific and standardized means, which is the mainstream of medical service.

Microservice architecture (MSA) is a kind of design style, which divides a large and complex application into microservices. Each microservice is developed independently and focuses only on accomplishing one task. In addition, the service has its own processing and lightweight communication mechanism, which can be deployed on one or multiple servers. With the advantages of rapid development, convenient maintenance, and easy management, MSA has been thoroughly studied in various scenarios in terms of architecture, demand and application [3]. The concept of divide and conquer is viewed as the core idea of MSA and has been investigated quite intensively in recent years. Each service can be deployed in the isolated execution environment immediately after testing to cope with multiple concerns and make each part of the decomposition highly cohesive and loosely coupled. However, the practice of MSA in the digital medical industry faces challenges of reliability.

The motivation of using MSA as HIS modeling is that, firstly, MSA is easy to expand. When the server processing capacity of a module is insufficient, add nodes in the application server of that module. In addition, after HIS services are migrated to the MSA, it is only necessary to ensure that the external contract remains unchanged, and the publishing process becomes particularly simple [4]. Because each service of HIS is independent and does not affect each other, it only needs to ensure that the external contract remains unchanged, and whatever language or framework to use internally. In MSA, because the impact between modules is isolated, the flexibility of reconfiguration is greatly increased.

This paper proposes a method to design the reliability in the field of public health. The main contributions are as follows:

(1) We first give the methodology of microservice reliability design for HIS and propose a modeling technology with Petri net to demonstrate the mechanism.

(2) We propose the composition strategies in response to the HIS reliability problems under fault conditions and prove the correctness of model and the effectiveness of strategies.

(3) We design the register management system in clinic aiming to verify the proposed model. The experimental results show that strategy improves service reliability.

The rest of the paper is organized as follows: Section 2 introduces related work. Section 3 shows the HIS reliability requirement design. Section 4 proposes the method. Section 5 demonstrates the verification and validation. Experimental results and discussion are in Section 6. Section 7 summarizes.

II. RELATED WORK

Microservice is a hot spot in the research of software services. Many scholars have proposed some related methods. Mayer and Weinreich provide an approach to extract and analyze the architecture of a microservices-based software system based on a combined static service information with runtime information [5]. A microservice self-adaptation description language MSDL specifies the self-adaptation logic at multiple levels and a supporting platform by extending

978-1-6654-2655-8/21/$31.00 ©2021 IEEE

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 08,2026 at 15:11:57 UTC from IEEE Xplore. Restrictions apply.

Kubernetes has been presented [6]. As in [7], an innovative automation mechanism to convert Fast healthcare Interoperability Resources (FHIR) resources into Artificial Intelligence Markup Language (AIML) files has been described, to facilitate the interaction and data gathering of medical and personal information.

To solve the problem in the estimation of MSA applications' reliability, the Enhanced Microservice Adaptive Reliability Testing (EMART) method has been presented, to faithfully assess the reliability of the microservice application in operation [8]. Reference [9] was the first to design MSAs for scalability and reliability in e-commerce. To address the fragility, poor flexibility, and hardware dependence of energy management systems, a microservice-based architecture has been introduced [10]. Compared with the service-oriented architecture, the established architecture can improve the load performance and scalability through fine-grained decomposition of system. Reference [11] describe a micro service composition problem in a multi cloud environment, which considers the service grouping relationship and corresponding correlation effects of service providers in or between clouds. The master microservice handles the creation of component configurations using a selection graph consisting of component tags and other descriptive attributes, as well as contextual information about the application user [12].

A hypothetical online movie subscription-based system has been designed using aspect-oriented modeling approach [13]. Camilli adopts a model-driven paradigm, which leverages solid foundation from Petri nets to specify and verify the behavior of time-dependent workflows [14]. Fan puts forward a systematic method to address the reliability, running time, and failure processing of resource scheduling in cloud computing to abstract the resource scheduling process as a meta-object and the Petri nets are used to construct the models [15]. Reference [16] presents a method of extending Petri net to support AOM. In this paper, the basic functions of the system are modeled as basic nets by Petri nets, and the crosscutting concerns are modeled as aspect nets. An adaptive resource allocation model has been proposed, which allocates users' jobs to the appropriate data center [17]. To provide real-time context aware operation based on data prediction flow processing, an extensible architecture has been proposed [18]. To date, less of the methods developed is perfect and many are far from mature to be used in public health area. How to seek balance between reliability is worthy of investigation.

III. MSRM FOR HIS BASED ON PRT NET

A. HIS microservice reliability design framework

HIS microservice reliability design can be modeled as the microservice reliability model (MSRM). The modeling and analysis can be seen as the Figure 1:

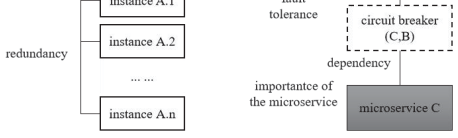


Fig. 1. Framework of microservice-based HIS reliability

This paper studies the dependency relationship between microservice and the microservice-based HIS reliability from three aspects, namely redundancy, fault tolerance, and degradation [19]. These compose of the reliability improvement strategies. The tools could be used to model the microservice instances, circuit breaker, and the reliability calculation for microservice is included in this section.

B. PrT net syntax and semantics

PrT net has been widely used to describe the concurrent and distributed system with loosely coupled features. Some basic concepts are introduced and others can refer to [20].

Definition 2. A 6-ary tuple $\Sigma=(N,IO,D,A_T,A_F,M_0)$ is called basic service net (BSN), where:

(1) $N=(P,T,F)$ is a basic Petri net, where P, T, F denote the finite set of place, transition and arc respectively.
(2) $IO \subset P$ is a special set, called Σ interface, which is represented by dotted circle.
(3) D is a non-empty finite individual set, and f_D and f_S are given as predicate sets *and* symbol sum sets respectively.
(4) A_T: $T \rightarrow f_D$, for $t \in T$, the free variable in $A_T(t)$ must be a directed arc free variable with t as one end.
(5) A_F: $F \rightarrow f_S$, if $(p,t) \in F$ or $(t,p) \in F$, then $A_F(p,t)$ or $A_F(t,p)$ is the sum of n-ary symbols and is null by default.
(6) M_0: $P \rightarrow f_S$ is the initial identification of Σ, $p \in P$, $M_0(p)$ does not contain any free variables.

BSN is mainly used to model the execution process of available microservice instances.

Definition 3. A 6-ary tuple $\Omega=(\Sigma,\Gamma,TI,TA,PI,PA)$ is called hierarchical service composition net (HSCN).

(1) Σ is a BSN, which describes the basic structure of Ω.
(2) $\Gamma=\{\Gamma_i | i \in N^*\}$ is a finite set of pages. Each page is a BSN or HSCN other than itself, and pages are disjoint. If Petri net $N_i=(P_i,T_i,F_i)$, then $\forall \Gamma_i,\Gamma_j \in \Gamma$, $(P_i \cup T_i \cup F_i) \cap (P_j \cup T_j \cup F_j) = \varnothing$.
(3) $TI \subset T$ is a collection of alternative nodes, where each page corresponds to a replacement node.
(4) $TA: TI \rightarrow \Gamma$ is the page allocation function, which is to assign specific pages for each replacement node.
(5) $PI \subset P$ is a set of port nodes which describe the input and output locations of the substitute nodes.
(6) PA is a port mapping function, which is used to map the port node of the substitute node to the input and output interface of the corresponding page, and each port node can only be mapped to one interface.

HSCN is mainly used for modeling components or the whole service composition process. Microservice MS_i is abstracted into individual $MS_i = (i, S_i, R_i, ES_i)$, where i is the i-th microservice while S_i, R_i, and ES_i represents the current state, reliability and significance of MS_i. $S_i \in \{n,an,de\}$ represents status of microservice, namely normal, abnormal, and degraded; R_i is the real number in the $[0,1)$ interval; $ES_i \in \{C,NC\}$. The microservice instance $MSI_{i,j}$ is abstracted into individual $MSI_{i,j} = (j,S_{i,j},R_{i,j})$, where j represents the position of available instance set, $DE(MS_i) = \{MSI_{i,1}, ...MSI_{i,j},... MSI_{i,m}\}$ and $S_{i,j}$, $R_{i,j}$ represent the current state and reliability of $MSI_{i,j}$. $S_{i,j} \in \{a,ua\}$

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 08,2026 at 15:11:57 UTC from IEEE Xplore. Restrictions apply.

represents status of microservice instance, namely available and unavailable; $R_{i,j}$ is the real number in the $[0,1)$ interval. Unless otherwise specified, individuals are abstracted as ϕ in HSCN. The input and output interface is marked with a superscript I and a superscript O, respectively [21].

C. Redundancy design for MSRM

The microservice with instances has been modeled by HSCN in Figure 2. The transition $t_{pred(ms,i)}$ and transition $t_{ms(i)}$ indicates the dependency relationship with the fire sequence.

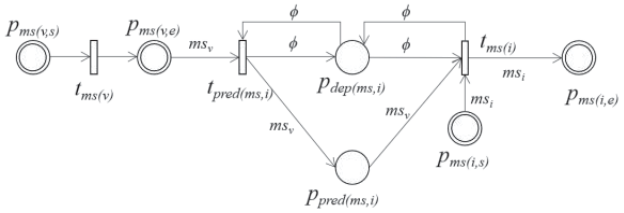


Fig. 2. Dependency relationship between microservices.

The microservice with its instances is transformed into the corresponding pages in HSCN, as shown in Figure 3.

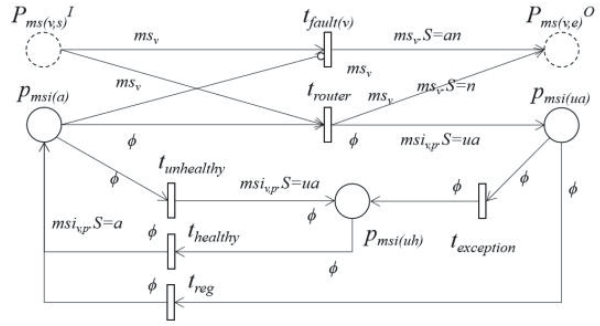


Fig. 3. MSRM with redundancy design.

The substitute nodes $p_{ms(v,s)}$ and $p_{ms(v,e)}$ are mapped into the page N_1 of component $P_{ms(v,s)}{}^I$ and $P_{ms(v,e)}{}^o$ in Figure 3. The data structure of token is $msi.S=\{a,ua\}$, a: status of msi is available; ua: status of msi is unavailable. For any healthy instance, transition $t_{healthy}$ fires and it changes to available, $msi_{v,p}.S=a$. Inversely, transition $t_{unhealthy}$ fires and its status becomes unavailable, $msi_{v,p}.S=ua$. The running instance can switch between healthy and unhealthy through transition $t_{exception}$ and transition t_{reg}. For any available registered instance, transition t_{router} fires and then it becomes unachievable with response to the others' request.

In case that the token in $p_{msi(a)}$ is empty, then transition $N_1.t_{fault(v)}$ fires, as shown by the inhibit arc in Figure 3, and the microservice becomes abnormal, $ms_v.S=an$. If $M(p_{msi(a)}) \neq \varnothing$, transition $N_1.t_{router}$ fires, the routed instance's status changes to unavailable, $msi_{v,p}.S=ua$ and ms_v status is normal, $ms_v.S=n$. The status and reliability of microservice depends on the number and status of its available instances. The basic elements in MSRM is decoupled with the page N_1; for instance the substitute nodes $p_{ms(i,s)}$ and $p_{ms(i,e)}$ can be mapped into elements in page N_1 similarly.

D. Fuse design for MSRM

The fusing mechanism is a measure to protect the usability of hospital service providers from the failure on the consumer side. Based on this principle, HSCN in Figure 4 and Figure 5 schematically illustrates the fusing mechanism.

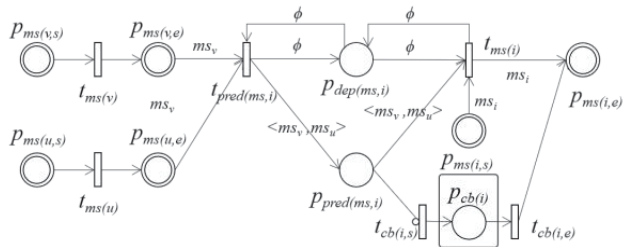


Fig. 4. MSRM with circuit breaker refinement (a).

The transition $t_{cb(i,s)}$ and $t_{cb(i,e)}$ are mapping to the transition $T_{cb(i,s)}{}^I$ and $T_{cb(i,e)}{}^O$ in the page respectively. As Figure 5 shows, when the fusible cutout is closed P_{closed}, the request is allowed to pass through. If the system health level is higher than threshold $(ms.S==an)\&\&(n<N)$, the switch will remain off p_{closed}. Otherwise, the cutout is on p_{opened}. When the fuse switch is on the p_{opened} position, request is forbidden. A surge of invalid request in a short time may be in fact detrimental to the others or drag the whole system down. When the fuse switch is in the open state p_{opened}, after the specified cycle T, the fuse will enter the semi-open state p_{semi}, and then the fuse will allow the request to pass again. When the request is successfully called t_{stoc}, the fuse will return to the closed state p_{closed}. If it fails t_{stoo}, it remains open p_{opened}. Token ms associated with former memorized fuse status gives result to current accessibility. Its data structure is formalized as $ms.S=\{n,an,de\}$, n: ms status is normal; an: ms status is abnormal; de: ms status is degraded.

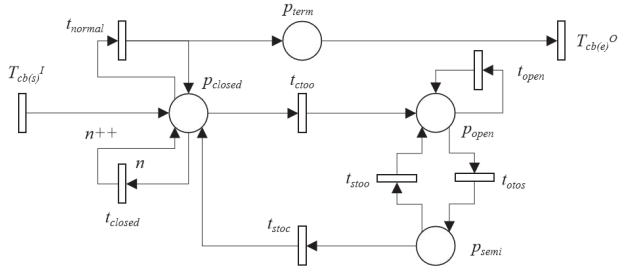


Fig. 5. MSRM with circuit breaker refinement (b).

The dependency relationship between medical services benefits the inter-connection of the system on one side, and on the other side it may cause the failures that the fault component will affect its dependency component. The failure of ms_u may cause the fault or degradation, which depends on significance, symbolized as ES with features of core and non-core. For instance, the $ms_u.S$ is abnormal and $ms_u.ES$ is non-core, then $ms_i.S$ is degraded. Degradation means the core function of microservice is available, and only some irrelevant parts are unavailable. Only when the predecessor microservice satisfies both "abnormal" and "core" conditions may the required microservice fail, which refines the HIS reliability design again. When the access peak of medical information system or some unimportant system goes down, degradation is considered to be an effective means to patient's accessibility.

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 08,2026 at 15:11:57 UTC from IEEE Xplore. Restrictions apply.

IV. VERIFICATION AND VALIDATION

A. Verification of model properties

In this section, attention has been drawn to verify the model's correctness and the efficiency of the proposed reliability strategies.

Property 1: The designed PrT net can effectively describe the reliability model of microservices.

Microservice and microservice instance are two basic elements in MSRM. The deployment function of microservice can be described by hierarchical PrT net. The status of microservice is decided by the instance's health status.

The prospect is to investigate the reliability tactics along with dependency relationship. In accordance with the fire rules, the microservice dependency in MSRM can be demonstrated in PrT net. Degradation refines the reliability coinciding with each status attribute of microservice. Structural fuse has been designed to fail at a prescribed threshold with fuse PrT net.

Property 2: Given that an instance is healthy, if and only if it is available and can be required by microservice.

Proof: This property can be proven by two steps. Firstly, it is to argue that only available microservice instance can be required. The second step is that, every microservice instance, with which status is available, can be required.

Step 1. $\forall msi_{u,p} \in MSI(ms_u)$, if $msi_{u,p}.S=ua$ and it is healthy, then it should be in place $p_{msi(ua)}$. There is no arc flow from $p_{msi(ua)}$ to t_{router}, so transition t_{router} cannot fire.

Step 2. $\forall msi_{u,p} \in MSI(ms_u)$, if $msi_{u,p}.S=a$, then in case that token is in place $p_{msi(uh)}$, transition $t_{healthy}$ fires; otherwise t_{reg} fires. So $msi_{u,p}$ should be in $p_{msi(a)}$, since $M(p_{msi(a)})\neq\varnothing$, t_{router} can fire definitely. Hence, every available instance can be required.

B. Validation of reliability strategies

The theoretical calculations have been conducted with and without strategy and to make a fair comparison.

Theorem 1: Given the precondition, that $ms_u \in pred(ms_i)$ and ms_u has failed, if ms_u is essential for ms_i, then redundancy strategy improves reliability.

Proof: Since ms_u is core function for ms_i, it is difficult to ignore failure. The reliability with non-redundant strategy is

$$R_{non-redundant} = msi_{u,p}.R \tag{1}$$

$msi_{u,p}$ is the p-th available instance of ms_u. The reliability with redundant strategy can be calculated by the equation:

$$R_{redundant} = 1 - \prod_{p=1}^{r} (1 - msi_{u,p}.R) \tag{2}$$

$msi_{u,p}.R$ is any available instance. In case that there exists only one backup, $R_{redundant}=R_{non\text{-}redundant}$, otherwise $R_{redundant}>R_{non\text{-}redundant}$, so redundant strategy helps to improve reliability.

The reliability of unhealthy instance $msi_{u,r}.R$ is equivalent to 0, so it makes no means for $R_{redundant}$, because element in Equation (1-$msi_{u,r}.R$) equals to 1.

Theorem 2: Given that $ms_u \in pred(ms_i)$, ms_u with non-core function failed, then degradation strategy improves reliability.

Proof: Given that ms_u is non-core function for ms_i, in case that ms_u fails, ms_i degrades instead of failing. Otherwise, ms_i works. Degradation means that core-function of microservice does not work, others are not influenced. So the Equation of $R_{degradation}$ and $R_{non\text{-}degradation}$ is calculated as:

$$AR_{non-degradation} = \prod_{p=1}^{ND} r_{u,p} \tag{3}$$

$$R_{degradation} = \prod_{p=1}^{D} r'_{u,p} \tag{4}$$

$r_{u,p}$ is the reliability of the p-th microservice instance of ms_u, $ms_u \in pred(ms_i)$ and ND is size of set predecessor. In Eq. (4), $r'_{u,p}$ is the reliability of the p-th microservice with non-core attributes and the set contain totally D elements. Because of $ND>=D$, $R_{degradation}>R_{non\text{-}degradation}$.

V. CASE STUDY

With the MSA, the outpatient clinic has been graphically demonstrated in Figure 6.

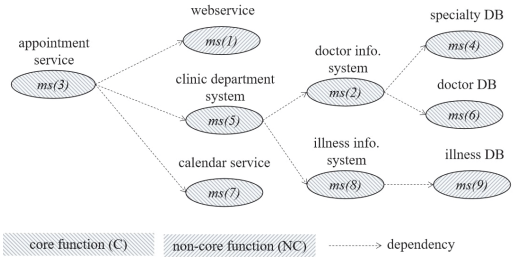


Fig. 6. Clinic register system with MSA.

A patient makes an appointment $ms(3)$ through access to the web-page $ms(3)$ with both clinic department system $ms(5)$, and calendar service $ms(7)$. The clinic department system $ms(5)$ depends on doctor information system $ms(2)$ and illness information system $ms(8)$, which relies on the data from specialty DB $ms(4)$ and doctor DB $ms(6)$ and data from illness DB $ms(9)$ respectively. The appointment web-service $ms(1)$ and the illness database $ms(9)$ are non-core function, whose failure may affect the system, but won't cause crash.

The experiment has been conducted with four groups according to the redundancy. Group 1 and 2 are non-redundant strategy. The first group is to select the instance at random, $ms.R(rand)$, and the second requests the instance with maximal reliability, $ms.R(max)$. As analyzed in the last section, the strategies are independent. So the fuse and degradation strategy has been applied in group 1 and group 2. Group 3 and 4 use the redundancy strategy with the redundant number of 5, and 10 to determine the reliability value. Like group 1 and group 2, these three groups can be composed with any one or both fuse and degradation strategies for the reliability value.

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 08,2026 at 15:11:57 UTC from IEEE Xplore. Restrictions apply.

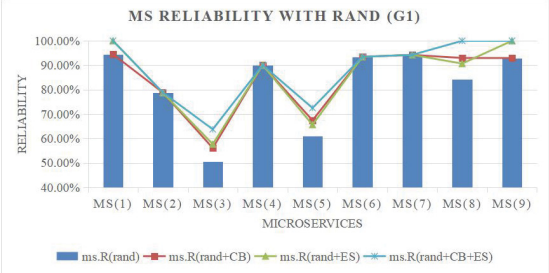


Fig. 7. Reliability of microservice in clinic with strategies from group 1.

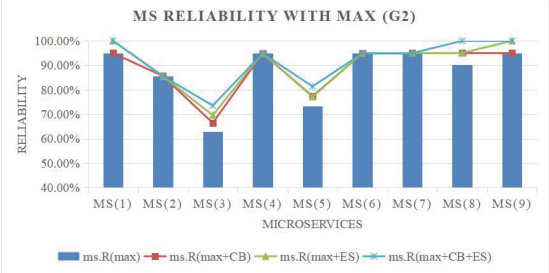


Fig. 8. Reliability of microservice in clinic with strategies from group 2.

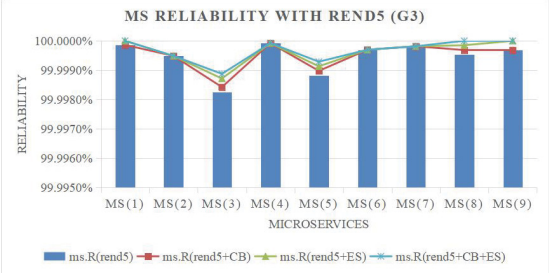


Fig. 9. Reliability of microservice in clinic with strategies from group 3.

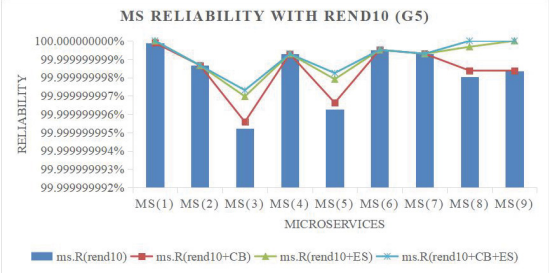


Fig. 10. Reliability of microservice in clinic with strategies from group 4.

Figure 7-10 shows that, the proposed composition strategy has effective functions based on the MSA design in clinic register system. The reliability with maximal reliability routing tactics is higher than random routing. The instance number does not affect the non-redundancy strategy because the microservice is routed randomly and the difference between random and maximal reliability is not huge. The microservice *ms*(*3*) and *ms*(*5*) in clinic system are with relatively lower reliability because of being more succeeding.

Figure 11-14 gives the gains of microservice in clinic register system on the composition strategies.

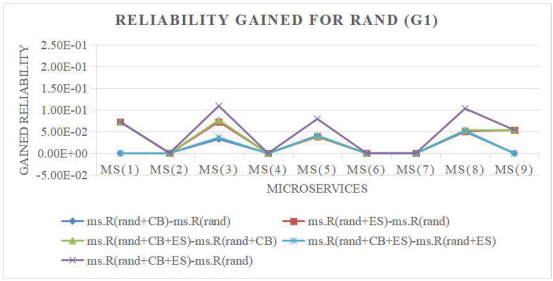


Fig. 11. Gained reliability of microservice with strategies from group 1.

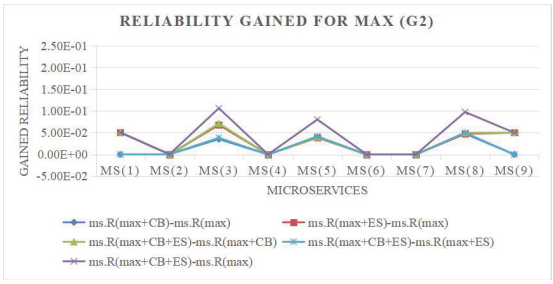


Fig. 12. Gained reliability of microservice with strategies from group 2.

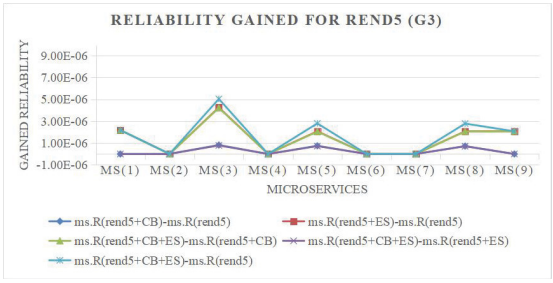


Fig. 13. Gained reliability of microservice with strategies from group 3.

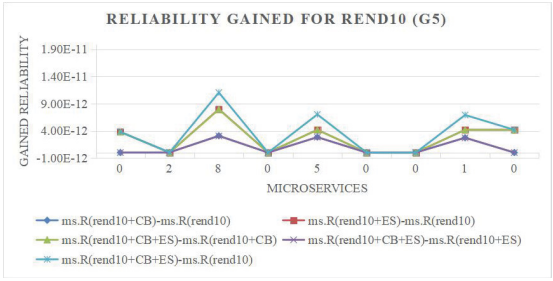


Fig. 14. Gained reliability of microservice with strategies from group 4.

The Gained reliability reaches about *2.50E-1* with single instance by using fuse and degradation strategies. With more and more redundancy, the effectiveness is about *7.0E-6*, *3.0E-11*, and *2.0E-13* with fuse and degradation strategies. So the order of magnitude shows that the effectiveness of redundancy is more obvious than the other two.

To conclude, the experimental reliability analysis procedure is given to illustrate the proposed technique, which shows a good fit to our goal.

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 08,2026 at 15:11:57 UTC from IEEE Xplore. Restrictions apply.

VI. Conclusion

Medical information management system carries out service, which is of great significance for the convenience of the patients to get medical treatment and improve the medical service. With HIS applied by MSA, microservices deploy these small modules by evolving the original monomer architecture into a smaller and more easily changed model. The main emphasis of the small and interconnected services is placed on high cohesion and low coupling. The redundancy strategy focuses on single service reliability tactic; the successor with fuse mechanism assists the whole system reliability by sacrificing its availability. The key idea is to generate the unavailability manually, which is controllable instead of being accidental; for the microservice with low reliability and irrelevant, the degradation can exercise its functions sufficiently.

Acknowledgments

This work is partially supported by the NSF of China, Shanghai Pujiang Talent Program.

References

[1] Sanchez-Gallegos, D. D., Galaviz-Mosqueda, A., Gonzalez-Compean, J. L., Villarreal-Reyes, S. , & Carretero, J. . On the Continuous Processing of Health Data in Edge-fog-cloud Computing by Using Micro/nanoservice Composition. IEEE Access, PP(99), 1-1, 2020.

[2] Gao, H. , Huang, W. Q. , Duan Y. C. . The Cloud-Edge Based Dynamic Reconfiguration to Service Workflow for Mobile Ecommerce Environments: A QoS Prediction Perspective. ACM Transactions on Internet Technology, Vol. 21, Issue. 1, pp 1–23, February 2021.

[3] Mishra, Arjit , S. Gupta , and S. Soni . "Designing Information System for Private Network using RBAC, FGAC and Micro service Architecture." International Journal of Engineering and Advanced Technology Vol. 10, Issue. 4, 2021, pp. 195-200.

[4] Zhou, X. , Peng, X. , Xie, T. , Sun, J. , & Ding, D. . Delta Debugging Microservice Systems with Parallel Optimization. IEEE Transactions on Services Computing, PP(99), 1-1, 2019.

[5] Khan, F. A. , Shaheen, S. , Asif, M. , Rahman, A. U. , Imran, M. , & Rehman, S. U. . Towards Reliable and Trustful Personal Health Record Systems: A Case of Cloud-dew Architecture based Provenance Framework. Journal of ambient intelligence and humanized computing, Vol. 10, Issue. 10, 2019, pp. 3795-3808.

[6] Mayer, B. , & Weinreich, R.c. An Approach to Extract the Architecture of Microservice-Based Software Systems, 12th IEEE International Symposium on Service-Oriented System Engineering (SOSE 2018), Vol. 1, pp. 21-30, IEEE Computer Society, 2018.

[7] Zhang, S. , Zhang, M. , Ni, L. , & Liu, P. . A Multi-Level Self-Adaptation Approach for Microservice Systems. 2019 IEEE 4th International Conference on Cloud Computing and Big Data Analysis (ICCCBDA), 2019, pp. 498-502.

[8] Pietrantuono, R. , Russo, S. , & Guerriero, A. . Testing Microservice Architectures for Operational Reliability. Software Testing, Verification and Reliability, Vol. 30, Issue. 2, March 2020.

[9] Asrowardi, I., Putra, S. D. , & Subyantoro, E. . Designing Microservice Architectures for Scalability and Reliability in E-commerce 2017 IEEE International Conference on Software Architecture Workshops (ICSAW), 2017, pp. 243-246.

[10] Lyu, Z. , Wei, H. , Bai, X. , & Lian, C. . Microservice-based Architecture for An Energy Management System. IEEE Systems Journal, vol. 14, no. 4, pp. 5061-5072, Dec. 2020.

[11] Gao M , Chen M , Liu A , et al. Optimization of microservice composition based on artificial immune algorithm considering fuzziness and user preference, IEEE Access, vol. 8, pp. 26385-26404, 2020.

[12] Mena M , Corral A , Iribarne L , et al. A Progressive Web Application Based on Microservices Combining Geospatial Data and the Internet of Things, IEEE Access, vol. 7, pp. 104577-104590, 2019.

[13] Hassan, S. , Ali, N. , & Bahsoon, R. . Microservice Ambients: An Architectural Meta-Modelling Approach for Microservice Granularity. 2017 IEEE International Conference on Software Architecture (ICSA), 2017, pp. 1-10.

[14] Camilli M. Continuous Formal Verification of Microservice-Based Process Flows. Software Architecture, 14th European Conference, ECSA 2020 Tracks and Workshops, September, 2020, Proceedings, Vol.1269, pp.420-435.

[15] Fan, G. , Yu, H. , & Chen, L. . A Formal Aspect-oriented Method for Modeling and Analyzing Adaptive Resource Scheduling in Cloud Computing. IEEE Transactions on Network and Service Management, vol. 13, no. 2, pp. 281-294, June 2016.

[16] Guan L , Xingyu L I , Hao H U , et al. A Petri Net-Based Approach for Supporting Aspect-Oriented Modeling. Frontiers of Computer Science in China, 2008, Vol. 2, Issue. 4, pp.413-423.

[17] Jung, G. , & Sim, K. M. . Agent-Based Adaptive Resource Allocation on the Cloud Computing Environment. International Conference on Parallel Processing Workshops, 2011, pp. 345-351.

[18] Ortiz G , Caravaca J A , Garcia-De-Prado A , et al. Real-Time Context-Aware Microservice Architecture for Predictive Analytics and Smart Decision-Making. IEEE Access, vol. 7, pp. 183177-183194, 2019.

[19] Calderón-Gómez H. et al., Telemonitoring System for Infectious Disease Prediction in Elderly People Based on a Novel Microservice Architecture, IEEE Access, vol. 8, pp. 118340-118354, 2020.

[20] Kim, T. H. , & Chang, C. K. . An Aspect-Oriented Approach to Resource Composition in Petri net-based Software Architectural Models," 30th Annual International Computer Software and Applications Conference (COMPSAC'06), 2006, pp. 87-94.

[21] Mallayya D , Ramachandran B . Aspect-Oriented Web Service Composition: A Petri Net Based Approach, International Conference on Cyber-enabled Distributed Computing & Knowledge Discovery, 2011, pp. 88-95.

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 08,2026 at 15:11:57 UTC from IEEE Xplore. Restrictions apply.